GANDAKI COLLEGE OF ENGINEERING AND SCIENCE

Lamachaur, 18, Kaski, Nepal

LAB REPORT OF

Agile software development

Lab1: Implementation of source code management and version control with git and github

Submitted By:	Submitted To:
Sandip Aryal Roll no: 35	Er.Rajindra Bdr. Thapa

1. Objective

The primary objective of this lab is to understand and apply the fundamental concepts of the Git Version Control System (VCS). The focus is on how Git facilitates core Agile principles such as iterative development, collaboration, and continuous integration by enabling developers to track changes, manage code history, and work together on a shared codebase.

2. Tools and Technologies

- **Git Bash:** A command-line interface for executing Git commands on Windows.
- **IDE / Text Editor:** A code editor (e.g., VS Code) for creating and modifying project files.
- **GitHub:** A web-based hosting service for Git repositories, providing a remote, centralized location for collaboration.

3. Theory

Version Control Systems are essential tools in modern software development, and their importance is amplified in an Agile environment. Agile methodologies thrive on rapid iteration, frequent releases, and seamless team collaboration.

Git, as a **Distributed Version Control System (DVCS)**, is uniquely suited for this. Unlike centralized systems where developers work on a single central server, Git gives each developer a complete local copy (a repository) of the project's history. This distributed nature allows for:

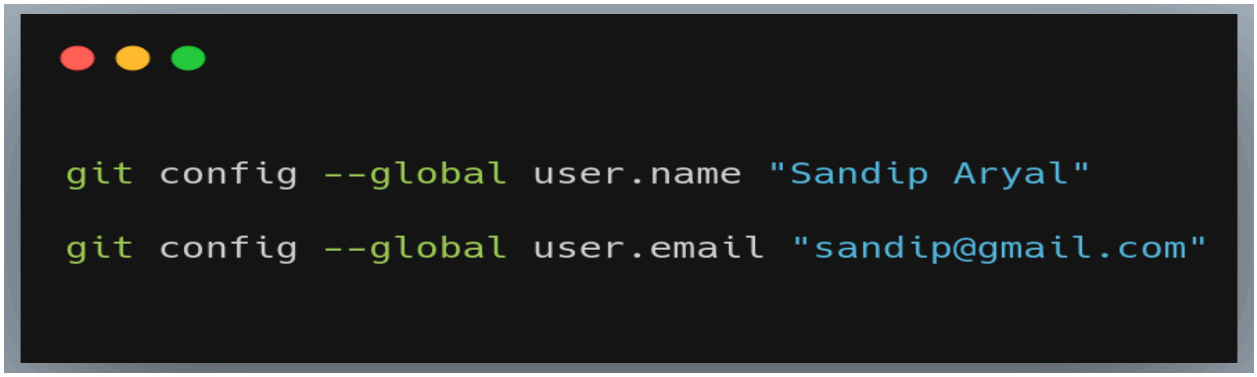
- **Parallel Development:** Developers can work independently and offline on features in their local repositories.
- **Feature Branching:** Teams can create separate branches for new features or bug fixes, isolating work without affecting the main, stable codebase.
- **Code Integrity:** Merging, pull requests, and a clear commit history ensure that changes are reviewed, tested, and integrated systematically, preserving the integrity of the project.

This lab demonstrates the basic workflow of using Git to manage a project, from local initialization to pushing changes to a remote repository on GitHub, simulating a foundational cycle in Agile development.

4. Implementation Steps and Observations

1. Initial Git Configuration

Before using Git for the first time on a new machine, we need to configure the username and email so the commits are associated to the correct person.



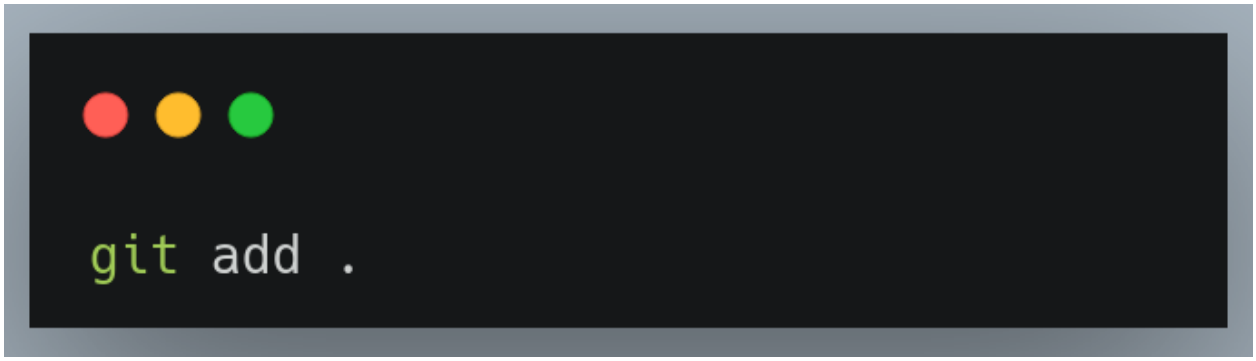
2. Initialize a Local Git Repository

To start tracking a project with Git, a repository must be initialized within the project's root directory.

Observation: Executing this command creates a hidden .git subdirectory. This directory contains all the necessary metadata for the repository, including the object database, commit history, and branch pointers.

3. Stage Changes for a Commit

Before committing changes, they must be added to the "staging area." This allows for granular control over which modifications are included in the next snapshot.



Alternatively, add specific files:

Observation: The git add command stages the modified files in the current working directory. The staging area acts as an intermediate step, allowing developers to selectively decide which changes to bundle into the upcoming commit.

4. Commit the Staged Changes

Once files are staged, they are permanently recorded in the local repository's history with a descriptive message.

Observation: This command takes the staged files and creates a commit object with a unique hash. The -m flag attaches a meaningful message to the commit, which is essential for project history readability and collaboration.

5. Connect to a Remote Repository

To collaborate or backup the code, the local repository is linked to a remote repository on GitHub.

First, create an empty repository on GitHub (e.g., my-project). Then, add it as a remote origin: bash

```
git remote add origin https://github.com/your-username/my-project.git
```

Observation: The remote add command links the local repository to the remote server. The name origin is the standard convention for referring to the primary remote repository.

6. Push Changes to the Remote Repository

To share the committed work with the team, the local commits are transferred to the remote repository. bash

```
git push -u origin main
```

Observation: The git push command uploads the local branch content to the remote repository. The -u flag sets the upstream tracking reference, allowing for simpler subsequent push and pull commands. This step makes the code accessible to other developers on the team, fulfilling the continuous integration requirement of Agile development.

5. Conclusion

This lab successfully demonstrated the foundational Git workflow essential for modern Agile software development. By performing local configuration, initializing a repository, staging, committing, and pushing to a remote host, the fundamental lifecycle of code tracking was replicated. Understanding these core commands and the distributed nature of Git is critical for enabling parallel development, maintaining code integrity, and ensuring seamless team collaboration across any project.